~ Complete Step by Step ~

JavaScript Notes

Basics to Advanced • All Topics Covered

Index

1 Introduction & Setup
2 Variables & Scope
3 Data Types & Conversion
4 Operators
5 Strings
6 Numbers & Math
7 Conditions (if, switch)
8 Loops
9 Functions (Part 1)
10 Functions (Part 2): Closures, this
11 Arrays (Part 1)
12 Arrays (Part 2): map, filter, reduce
13 Objects
14 Date, Set, Map, JSON
15 DOM (Part 1)
16 DOM Events (Part 2)
17 Browser Object Model & Storage
18 Error Handling & Debugging
19 Asynchronous JavaScript
20 Fetch API & AJAX
21 ES6+ Modern Features
22 Prototypes & OOP
23 Classes
24 Modules, Iterators, Generators
25 Regular Expressions
26 Advanced Concepts
27 Web APIs
28 Node.js, npm & Tools
29 Mini Projects
30 Interview Q&A (2 pages)
31 Cheat Sheet

How to use

→ Learn HTML and CSS first. JavaScript brings them to life.
→ *keywords* = orange • "strings" = green • numbers = pink • // comments = purple.
→ Open the browser console (F12) and type every example yourself!

Notes by asadokz18 ✿

1. Introduction & Setup

- JavaScript (JS) is a high-level, interpreted, dynamic programming language that makes web pages **interactive**.
- Created by **Brendan Eich** in **10 days** (1995, Netscape). Standard name: **ECMAScript (ES)**. Java ≠ JavaScript – totally different languages!
- Runs in the **browser** (Chrome V8, Firefox SpiderMonkey) and on servers with **Node.js**.
- Single-threaded, **event-driven**, supports OOP, functional style and async programming.

What can JavaScript do?

- Change HTML content & CSS styles
- Respond to clicks, typing, scrolling
- Validate forms
- Get data from servers (AJAX / fetch)
- Build web apps (React, Vue, Angular)
- Server side (Node.js), mobile (React Native)
- Games, animations, charts
- Desktop apps (Electron)

ECMAScript versions

Year	Version	Main features
1997 / 2009	ES1 … ES5	Basics, JSON, strict mode
2015	**ES6 / ES2015**	let, const, arrow functions, classes, promises, modules, template strings
2016 – 2019	ES7 – ES10	** operator, async/await, Object.entries, flat()
2020 – now	ES11+	?. and ??, BigInt, replaceAll, at(), structuredClone, Object.groupBy

How to run JavaScript

```
<!-- 1. Internal: inside HTML -->
<script>
  console.log("Hello, World!");
</script>

<!-- 2. External file (best) - put before </body> or use defer -->
<script src="app.js" defer></script>
```

- **Browser console:** press **F12** → Console tab. **Node.js:** ***node app.js*** in terminal.

Output methods

```
console.log("Hello");                  // show in console (debugging)
alert("Hi!");                          // popup box
document.write("Hello page");          // write into page (avoid)
document.getElementById("demo").innerHTML = "Hello";   // change HTML element
console.error("Error!"); console.warn("Warning"); console.table([1,2]);
```

Syntax basics

- **Statements** end with **;** (optional but recommended). Code runs **top to bottom**.
- **Case-sensitive:** name ≠ Name. Use **camelCase**: myFirstName.
- Comments: ***// single line*** and ***/* multi-line */***. Whitespace is ignored.
- **Strict mode:** ***"use strict";*** at the top catches common mistakes.

2. Variables & Scope

A **variable** is a named box that stores data. Declare with **var**, **let** or **const**.

```
let name = "Asad";      // can be changed later
const PI = 3.14159;     // cannot be reassigned
var old = 10;           // old way (function scope) - avoid
let age;                // declared, value is undefined
let a = 1, b = 2, c = 3;   // many in one line
name = "Ali";           // OK for let,   PI = 3 → TypeError for const
```

	var	let	const
Scope	Function	Block { }	Block { }
Re-assign	Yes	Yes	No
Re-declare	Yes	No	No
Hoisted	Yes (undefined)	Yes (TDZ error)	Yes (TDZ error)
Must init	No	No	Yes

Rule

Use **const** by default. Use **let** only if the value changes. **Avoid var.** Note: const object / array contents **can still be changed** (only the reference is fixed).

Naming rules

- Start with a **letter, _ or $** - never a digit. Only letters, digits, _ and $. No spaces. Cannot be a reserved word (let, class, return…).
- Style: **camelCase** (userName), **UPPER_CASE** for constants (MAX_SIZE), **PascalCase** for classes. Use meaningful names!

Scope – where a variable can be used

```
let g = "global";                 // Global scope: visible everywhere

function show() {
  let local = "function scope";   // visible only inside this function
  if (true) {
    let block = "block scope";    // visible only inside { }
    var leaky = "var ignores block";
  }
  console.log(leaky);   // works
  console.log(block);   // ReferenceError
}
```

Hoisting & TDZ

```
console.log(x);   // undefined  (var is hoisted, value is not)
var x = 5;
console.log(y);   // ReferenceError: Cannot access 'y' before initialization (Temporal Dead Zone)
let y = 5;
sayHi();          // works → function declarations are fully hoisted
function sayHi() { console.log("Hi"); }
```

Warning

Assigning without declaring (**z = 5**) creates a **global** variable - bug! Strict mode forbids it.

Shadowing

An inner variable with the same name hides the outer one inside its scope.

3. Data Types & Conversion

JavaScript is **dynamically typed** – a variable can hold any type and change type later.

7 Primitive types

- **String** – "text"
- **Number** – 10, 3.14, NaN, Infinity
- **BigInt** – 123n
- **Boolean** – true / false
- **undefined** – not assigned
- **null** – intentionally empty
- **Symbol** – unique id

Non-primitive (reference)

- **Object** – {name: "Ali"}
- **Array** – [1, 2, 3]
- **Function**
- **Date, RegExp, Map, Set...**

Primitives are copied **by value**. Objects are copied **by reference**.

typeof operator

```
typeof "hi"        // "string"      typeof 42          // "number"
typeof true        // "boolean"     typeof undefined   // "undefined"
typeof null        // "object"  ← famous bug!   typeof 10n   // "bigint"
typeof [1,2]       // "object"  (use Array.isArray())   typeof function(){}  // "function"
typeof Symbol()    // "symbol"      typeof {}          // "object"
```

Special values	Meaning
undefined	Variable declared but no value; missing property; function returns nothing.
null	Empty value set on purpose.
NaN	"Not a Number" (0/0, "abc"*2). Check with **Number.isNaN(x)**. NaN !== NaN!

Truthy & Falsy

Falsy (8): **false, 0, -0, 0n, "" , null, undefined, NaN** · Everything else is **truthy** (including "0", [], {}, "false").

Type conversion

```
// Explicit (manual)
Number("42")  // 42      Number("abc") // NaN      Number("")  // 0      Number(true) // 1
parseInt("42px") // 42    parseFloat("3.14abc") // 3.14
String(123)   // "123"   (123).toString()  // "123"      Boolean("hi") // true
// Implicit (automatic coercion)
"5" + 2       // "52"  (+ with string = concatenation)
"5" - 2       // 3     (- * / convert to number)
true + 1      // 2       [] + {} // "[object Object]"
```

Careful

Use === (strict equal) instead of == to avoid surprises: **0 == "0"** is true but **0 === "0"** is false.

4. Operators

Arithmetic

+	-	*	/	%	**	++	--
add	subtract	multiply	divide	remainder 10%3=1	power 2**3=8	increment	decrement

```
let i = 5;
i++;   // 6 (post)   ++i  // pre-increment   → let a = i++ (a gets old value),  let b = ++i (new value)
"Hello " + "World"   // string concatenation → "Hello World"
```

Assignment

= += -= *= /= %= **= &&= ||= ??= – e.g. **x += 5** means x = x + 5.

Comparison (result is true / false)

==	===	!=	!==	>	<	>=	<=
equal (value)	**strict equal (value + type)**	not equal	**strict not equal**	greater / less than (or equal)			

Logical

```
true && false   // false  AND (both true)       true || false  // true  OR (any true)       !true  // false  NOT
// Short-circuit usage
let user = name || "Guest";       // first truthy value
isLogged && showMenu();           // run only if first is true
let v = 0 ?? 10;                  // 0   → ?? (nullish) only checks null / undefined
let w = 0 || 10;                  // 10  → || treats 0 as falsy
```

Ternary (conditional) & optional chaining

```
let result = age >= 18 ? "Adult" : "Minor";      // condition ? ifTrue : ifFalse
let city = user?.address?.city;                  // undefined instead of an error if missing
let fn = obj.method?.();                         // call only if it exists
```

Other operators

Operator	Use
typeof / instanceof	Type of value / is object created by a class? **[] instanceof Array**
in / delete	**"name" in obj · delete obj.age**
... (spread / rest)	Expand or collect values: **[...a, ...b]**
, (comma) · void · new	Many expressions · returns undefined · create object
& | ^ ~ << >> >>>	Bitwise operators (work on 32-bit binary numbers)

Precedence (high → low)

( ) → ++ -- ! typeof → ** → * / % → + - → < > <= >= → == === → && → || ?? → ?: → = . Use brackets when unsure!

5. Strings

Text inside **"double"**, **'single'** quotes or **`backticks`**. Strings are **immutable** (cannot be changed, methods return a new string).

```
let s = "Hello, World";
s.length          // 12          s[0] // "H"          s.at(-1) // "d" (last char)
"It's" ; 'He said "Hi"' ; "Line1\nLine2"          // escapes: \n new line, \t tab, \\ backslash, \' \"
```

Template literals (backticks) ✔

```
let name = "Asad", age = 20;
let msg = `My name is ${name} and next year I will be ${age + 1}.`;    // interpolation
let multi = `Line 1
Line 2`;                                                                // multi-line
```

Common string methods

Method	What it does
toUpperCase() / toLowerCase()	Change case
trim() trimStart() trimEnd()	Remove spaces from ends
slice(start, end)	Extract part (negative ok): "Hello".slice(1,3) → "el"
substring(s, e) • substr()	Like slice (no negatives) • substr deprecated
indexOf() lastIndexOf()	Position of text (-1 if not found)
includes() startsWith() endsWith()	Return true / false
replace(a,b) • replaceAll(a,b)	Replace first / all matches (regex allowed)
split(sep)	String → array: "a,b,c".split(",") → ["a","b","c"]
repeat(n) • concat()	"ab".repeat(3) → "ababab"
padStart(len, ch) padEnd()	"5".padStart(3,"0") → "005"
charAt(i) • charCodeAt(i)	Character / its Unicode number
match() matchAll() search()	Find with regular expression
localeCompare() • normalize()	Compare for sorting • Unicode normalize

```
let t = "  JavaScript is fun  ";
t.trim().toUpperCase()          // "JAVASCRIPT IS FUN"
"apple,banana".split(",")       // ["apple", "banana"]
"I love cats".replace("cats", "dogs")   // "I love dogs"
"abc".includes("b")             // true
[..."hey"].reverse().join("")   // "yeh"  → reverse a string
```

Compare

Strings compare by Unicode: **"a" < "b"** true, **"Z" < "a"** true.

Remember

s.toUpperCase() doesn't change s → store it: **s = s.toUpperCase()**

6. Numbers & Math

- All numbers are **64-bit floating point** (no separate int / float). Safe integer range: ±9,007,199,254,740,991 (**Number.MAX_SAFE_INTEGER**). Bigger → use **BigInt** (**123n**).
- Formats: **10**, **3.14**, **1e6** (1,000,000), **0xFF** hex, **0b101** binary, **0o17** octal, **1_000_000** separator.

```
0.1 + 0.2              // 0.30000000000000004  ← floating-point error!
(0.1 + 0.2).toFixed(2)  // "0.30"  (returns a string)   → +(0.1+0.2).toFixed(2)
1 / 0   // Infinity     -1 / 0 // -Infinity      "abc" * 2  // NaN
```

Number methods & properties

Method	Use
Number(x) · **parseInt(x, 10)** · **parseFloat(x)**	Convert to number
num.toFixed(2) · **toPrecision(4)**	Fixed decimals / total digits (return strings)
num.toString(2)	Number → string in base (2 = binary, 16 = hex)
Number.isInteger() · **isFinite()**	Checks (true / false)
Number.isNaN() · **isSafeInteger()**	NaN check (better than global isNaN)
toLocaleString()	Format: (1234567.8).toLocaleString("en-US") → "1,234,567.8"
Number.MAX_VALUE · **MIN_VALUE** · **EPSILON**	Constants

Math object (no new needed)

Method	Result
Math.round(4.6) · **floor(4.9)** · **ceil(4.1)** · **trunc(4.9)**	5 · 4 (down) · 5 (up) · 4 (remove decimal)
Math.abs(-7) · **sign(-3)**	7 · -1
Math.max(1,5,3) · **Math.min(...)**	5 · smallest (use **Math.max(...arr)**)
Math.pow(2,3) · **Math.sqrt(16)** · **cbrt()**	8 · 4 · cube root
Math.random()	Random decimal 0 ≤ n < 1
Math.PI · **Math.E** · **sin() cos() log()...**	3.14159... · 2.718... · trigonometry / logs

Random numbers

```
Math.floor(Math.random() * 10)              // 0 to 9
Math.floor(Math.random() * 10) + 1          // 1 to 10
function randomInt(min, max) {              // min to max (both included)
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

Money tip

Because of floating errors, store money as **integers** (cents) or round with **toFixed**.

7. Conditions (Decision Making)

if ... else if ... else

```
let marks = 72;
if (marks >= 90) {
  console.log("A+");
} else if (marks >= 70) {
  console.log("B");          // this runs
} else if (marks >= 50) {
  console.log("C");
} else {
  console.log("Fail");
}
if (marks > 40) console.log("Pass");        // braces optional for one statement
```

Flow

Condition is checked → if **true** run that block → if **false** go to next else if / else. Only **one block** runs.

Truthy / falsy

if (name), ***if (arr.length)***, ***if (!user)*** work with truthy and falsy values.

Nested if & logical operators

```
if (age >= 18 && hasId) { console.log("Entry allowed"); }
if (day === "Sat" || day === "Sun") { console.log("Weekend"); }
if (!(x > 5)) { /* x is 5 or less */ }
if (user) { if (user.isAdmin) { console.log("Admin"); } }     // nested → user?.isAdmin
```

switch statement

Compares one value with many cases using **strict equality (===)**.

```
let day = 3, name;
switch (day) {
  case 1:
    name = "Monday"; break;
  case 2:
    name = "Tuesday"; break;
  case 6:
  case 7:                       // grouped cases (fall-through)
    name = "Weekend"; break;
  default:
    name = "Midweek day";       // runs if nothing matches
}
```

Don't forget break!

Without **break** the code **falls through** and runs the next cases too. ***default*** is optional and can be anywhere.

Ternary & short-circuit

```
let status = age >= 18 ? "Adult" : "Minor";
let grade = m >= 80 ? "A" : m >= 60 ? "B" : "C";   // chained ternary (keep it short!)
let name2 = input || "Anonymous";
let count = data?.items?.length ?? 0;
```

When to use what

if/else → ranges & complex conditions · switch → one value vs many fixed choices · ternary → short either/or values · object lookup ***colors[key] ?? "#000"*** → replaces long switch.

8. Loops

Loops repeat code many times. Don't write the same line again and again!

1) for loop

```
for (let i = 0; i < 5; i++) {       // (init; condition; update)
  console.log(i);                   // 0 1 2 3 4
}
for (let i = 10; i > 0; i -= 2) { }   // counting down by 2
```

Steps: ① init runs once ② condition checked ③ body runs ④ update runs ⑤ repeat from ②. Any part may be empty: ***for (;;)*** = infinite loop.

2) while loop & 3) do...while

```
let n = 1;
while (n <= 3) {   // check first
  console.log(n);
  n++;
}
```

```
let k = 10;
do {               // run at least once
  console.log(k);
  k++;
} while (k < 5);   // prints 10
```

4) for...of – loop over VALUES (arrays, strings, Set, Map)

```
for (const fruit of ["apple", "mango"]) console.log(fruit);
for (const ch of "Hi") console.log(ch);                          // "H" "i"
for (const [i, v] of ["a","b"].entries()) console.log(i, v);  // index + value
```

5) for...in – loop over KEYS (objects)

```
const user = { name: "Asad", age: 20 };
for (const key in user) console.log(key, user[key]);   // name Asad, age 20
```

Don't

Use **for...in on arrays** (gives string indexes and inherited keys). Use **for...of** or array methods.

break, continue & labels

```
for (let i = 1; i <= 10; i++) {
  if (i === 3) continue;       // skip this round
  if (i === 6) break;          // exit the loop completely
  console.log(i);              // 1 2 4 5
}
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer;   // label: jump to the outer loop
  }
}
```

Array loop methods & patterns

- ***arr.forEach(fn)***, ***map***, ***filter***, ***reduce*** (chapter 12). Nested loops for tables / patterns.
- Sum: ***let sum = 0; for (const n of nums) sum += n;*** · Table: ***for (i=1;i<=10;i++) console.log(`5 x ${i} = ${5*i}`)***

Avoid infinite loops

Make sure the condition becomes false (update the variable!).

9. Functions – Part 1

A **function** is a reusable block of code that performs a task. Write once, use many times (**DRY**).

Ways to create a function

```
// 1. Function declaration (hoisted - can call before definition)
function add(a, b) {
  return a + b;
}
// 2. Function expression (not hoisted)
const multiply = function (a, b) { return a * b; };
// 3. Arrow function (ES6) - short syntax, no own `this`
const sub = (a, b) => a - b;            // implicit return
const square = x => x * x;              // one param: () optional
const hello = () => console.log("Hi");
const getUser = () => ({ name: "Ali" });   // return object → wrap in ( )
const big = (a, b) => { const s = a + b; return s * 2; };   // block body needs return
```

Call it: **add(2, 3)** → 5. Without ( ) you only get the function itself.

Term	Meaning
Parameter	Variable in the definition: **function f(a, b)**
Argument	Actual value passed when calling: **f(2, 3)**
return	Sends a value back and **stops** the function. No return → undefined.

Default, rest & spread parameters

```
function greet(name = "Guest", greeting = "Hello") { return `${greeting}, ${name}!`; }
greet();              // "Hello, Guest!"       greet("Ali");  // "Hello, Ali!"

function sum(...nums) {                 // rest: collects all extra args into an array
  return nums.reduce((t, n) => t + n, 0);
}
sum(1, 2, 3, 4);                        // 10
Math.max(...[4, 9, 2]);                 // spread: array → separate arguments
```

- Missing arguments become **undefined**; extra arguments are ignored (still available in **arguments** object – not in arrows).
- **Primitives** are passed by value, **objects / arrays** by reference (changes inside affect the original).

Callbacks & higher-order functions

```
function process(a, b, callback) { return callback(a, b); }   // function passed as argument
process(4, 5, (x, y) => x * y);                                // 20
setTimeout(() => console.log("later"), 1000);                  // classic callback
function makeAdder(n) { return x => x + n; }                       // returns a function
makeAdder(5)(10);                                              // 15
```

IIFE & recursion

```
(function () { console.log("runs immediately"); })();      // IIFE: Immediately Invoked Function Expression
(() => console.log("arrow IIFE"))();

function factorial(n) { return n <= 1 ? 1 : n * factorial(n - 1); }   // function calls itself
factorial(5);   // 120  → needs a base case or you get "Maximum call stack size exceeded"
```

10. Functions – Part 2 (Closures, this...)

Closures

A **closure** = a function that remembers the variables of the place where it was created, even after that outer function has finished.

```
function counter() {
  let count = 0;                  // private variable
  return function () { return ++count; };
}
const c1 = counter();
c1(); c1(); c1();                 // 1, 2, 3   ← count is remembered
const c2 = counter();  c2();      // 1         ← a separate closure
```

- Uses: **data privacy**, factories, memoization, callbacks, partial application, event handlers.
- Classic trap: **var** in a loop with setTimeout prints the same value – **let** fixes it (new binding per round).

The this keyword

Where	this =
Global / normal function	window (browser) · undefined in strict mode
Method call **obj.fn()**	**obj** (the object before the dot)
Arrow function	Takes **this** from the surrounding code (lexical)
Constructor **new F()** / class	The new object
Event handler (normal function)	The element that got the event
call / apply / bind	Whatever you set

call(), apply(), bind()

```
function intro(city, country) { return `${this.name} from ${city}, ${country}`; }
const p = { name: "Asad" };
intro.call(p, "Lahore", "PK");          // call: args one by one
intro.apply(p, ["Lahore", "PK"]);       // apply: args in an array
const bound = intro.bind(p, "Lahore"); bound("PK");    // bind: returns a NEW function, runs later
```

More function concepts

```
// Pure function: same input → same output, no side effects
const double = n => n * 2;
// Currying: function that returns functions
const add = a => b => c => a + b + c;      add(1)(2)(3);    // 6
// Function composition
const compose = (f, g) => x => f(g(x));
// Memoization (cache results with a closure)
function memo(fn) { const cache = {}; return n => cache[n] ?? (cache[n] = fn(n)); }
// Functions are objects → can have properties
double.description = "doubles a number";
```

First-class functions

Functions can be stored in variables, passed as arguments, returned and put in objects.

Arrow limits

No own **this**, **arguments**; can't be used with **new**; not good as object methods.

11. Arrays – Part 1

An **array** is an ordered list of values. Index starts at **0**. Can hold any types. Arrays are objects.

```
const fruits = ["apple", "mango", "banana"];
const mixed = [1, "two", true, null, [5, 6], { a: 1 }];
const empty = [];   const big = new Array(5);   const nums = Array.from({length: 3}, (_, i) => i);   // [0,1,2]
fruits[0]          // "apple"          fruits.at(-1)  // "banana" (last)
fruits.length      // 3                fruits[1] = "kiwi";      // change item
Array.isArray(fruits)   // true        mixed[4][1]  // 6 (nested)
```

Add / remove items (change the original array)

Method	Does
push(x)	Add to **end**; returns new length
pop()	Remove **last**; returns it
unshift(x)	Add to **start**
shift()	Remove **first**; returns it
splice(start, delCount, ...items)	Remove / insert / replace anywhere
sort() · reverse()	Sort / reverse **in place**
fill(v) · copyWithin()	Fill with a value

```
let a = [1, 2, 3];
a.push(4);          // [1,2,3,4]        a.pop();          // [1,2,3]
a.unshift(0);       // [0,1,2,3]        a.shift();        // [1,2,3]
a.splice(1, 1);          // remove 1 item at index 1 → [1,3]
a.splice(1, 0, "x", "y");   // insert at index 1 → [1,"x","y",3]
a.splice(0, 1, "new");       // replace
```

Non-mutating methods (return new / info)

Method	Does
slice(start, end)	Copy a part: [1,2,3,4].slice(1,3) → [2,3]
concat() · [...a, ...b]	Join arrays
join(sep)	Array → string: ["a","b"].join("-") → "a-b"
indexOf(x) · lastIndexOf(x)	Position or -1
includes(x)	true / false
flat(depth) · toString()	Flatten nested arrays · to string
toSorted() toReversed() toSpliced() with()	New (ES2023) non-mutating versions

Sorting

```
["b","a","c"].sort()                    // ["a","b","c"]  (default: as STRINGS!)
[10, 9, 1, 100].sort()                  // [1, 10, 100, 9]  ← wrong for numbers
[10, 9, 1, 100].sort((a, b) => a - b)       // [1, 9, 10, 100]  ascending      (b - a = descending)
users.sort((x, y) => x.name.localeCompare(y.name));    // sort objects by text
```

Copy arrays

let b = a only copies the **reference**! Real copy: **[...a]** or **a.slice()** (shallow) · deep: **structuredClone(a)**.

12. Arrays – Part 2 (Higher-order methods)

These methods take a **callback function** and do **not change** the original array (except forEach side effects).

Method	Returns	Use
forEach(fn)	nothing	Do something for each item
map(fn)	new array (same length)	Transform every item
filter(fn)	new array (fewer)	Keep items where fn is true
reduce(fn, init)	single value	Sum, count, group, flatten…
find(fn) · findIndex(fn)	first item / index	First match (undefined / -1)
findLast · findLastIndex	last item / index	Search from the end
some(fn)	true / false	Is **at least one** item true?
every(fn)	true / false	Are **all** items true?
flatMap(fn)	new array	map then flat(1)

```
const nums = [1, 2, 3, 4, 5];
nums.forEach((n, i) => console.log(i, n));
nums.map(n => n * 2)                    // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0)           // [2, 4]
nums.reduce((total, n) => total + n, 0) // 15   (accumulator, current, initial value)
nums.find(n => n > 3)                   // 4        nums.findIndex(n => n > 3)  // 3
nums.some(n => n > 4)                   // true     nums.every(n => n > 0)      // true
[[1, 2], [3, [4]]].flat(Infinity)       // [1, 2, 3, 4]
```

Chaining & real example

```
const total = products
  .filter(p => p.inStock)               // only available
  .map(p => p.price * p.qty)            // line totals
  .reduce((sum, x) => sum + x, 0);      // add all

const byAge = users.reduce((groups, u) => {       // group by property
  (groups[u.age] ||= []).push(u);
  return groups;
}, {});
```

Destructuring & spread with arrays

```
const [first, second, ...others] = [10, 20, 30, 40];   // first=10, second=20, others=[30,40]
const [a, , c = 5] = [1, 2];                            // skip items, default values
[a, c] = [c, a];                                        // swap variables
const copy = [...nums];   const merged = [...nums, 6, 7];
const unique = [...new Set([1, 1, 2, 3, 3])];           // [1, 2, 3]  remove duplicates
```

Other useful

Array.of() · Array.from(iterable) · keys() values() entries() · Object.groupBy() · at()

Remember

map = transform, filter = select, reduce = combine. Always **return** a value in map/filter/reduce callbacks.

13. Objects

An **object** stores data as **key : value** pairs (properties). Methods are properties that are functions.

```
const user = {
  name: "Asad",
  age: 20,
  "favourite color": "blue",          // key with space needs quotes
  hobbies: ["coding", "cricket"],
  address: { city: "Lahore" },        // nested object
  greet() { return `Hi, I am ${this.name}`; }     // method (shorthand)
};
user.name                 // "Asad"        (dot notation)
user["favourite color"] // "blue"       (bracket notation - for dynamic / special keys)
user.address.city       // "Lahore"       user.greet()   // "Hi, I am Asad"
user.email = "a@b.com";   delete user.age;   // add / delete
"name" in user   // true     user.hasOwnProperty("name")   Object.hasOwn(user, "name")
```

Useful Object methods

Method	Does
Object.keys(o) · values(o)	Array of keys / values
Object.entries(o)	Array of [key, value] pairs ↔ **Object.fromEntries(arr)**
Object.assign(target, src)	Copy / merge properties (shallow)
Object.freeze(o) · isFrozen()	Make completely read-only
Object.seal(o)	No add / delete, values can change
Object.create(proto)	New object with given prototype
Object.defineProperty(o, k, desc)	Control writable / enumerable / getter
structuredClone(o)	Deep copy (copy of nested too)

```
for (const [key, value] of Object.entries(user)) console.log(key, value);
const copy = { ...user, age: 21 };              // spread: copy and override
const merged = { ...a, ...b };                  // later wins
const { name, address: { city }, role = "user" } = user;   // destructuring (+ nested + default)
const key = "score";  const o = { [key]: 99, name };       // computed key, shorthand property
```

Getters / setters & this

```
const person = { first: "A", last: "K",
  get full() { return `${this.first} ${this.last}`; },
  set full(v) { [this.first, this.last] = v.split(" "); } };
person.full = "Asad Ullah";   console.log(person.full);
```

Reference!

const b = a → same object. **{ ...a }** = shallow copy only.

Optional chaining

user?.address?.city avoids errors on missing parts.

14. Date, Set, Map & JSON

Date object

```
const now = new Date();                          // current date & time
const d1 = new Date("2026-10-15");               const d2 = new Date(2026, 9, 15, 10, 30);   // month 0-11! (9 = October)
now.getFullYear()  now.getMonth()  now.getDate()  // year, month (0-11), day of month
now.getDay()   // 0 = Sunday      now.getHours() getMinutes() getSeconds() getMilliseconds()
now.getTime()  // ms since 1 Jan 1970   Date.now()   // same, current time
d1.setDate(d1.getDate() + 7);                    // add 7 days  (also setFullYear, setMonth, setHours...)
now.toISOString()   now.toLocaleDateString("en-GB")   now.toLocaleTimeString()   now.toDateString()
const diffDays = (d2 - d1) / (1000 * 60 * 60 * 24);   // date difference
```

Set – unique values

```
const s = new Set([1, 2, 2, 3]);    // {1, 2, 3}
s.add(4); s.has(2); s.delete(1); s.size; s.clear();
for (const v of s) console.log(v);
[...new Set(arr)]                   // remove duplicates from array
```

Map – key/value with ANY type of key

```
const m = new Map();
m.set("name", "Asad").set(1, "one").set(obj, "object key");
m.get("name");  m.has(1);  m.delete(1);  m.size;
for (const [k, v] of m) console.log(k, v);       m.forEach((v, k) => {});
const fromObj = new Map(Object.entries({ a: 1 }));
```

	Object	Map
Keys	strings / symbols	any type
Order	mostly insertion	always insertion order
Size	Object.keys(o).length	map.size

WeakSet / WeakMap – hold objects weakly (garbage collected when no other reference); not iterable.

JSON – JavaScript Object Notation

➜ Text format to store / send data. Keys **must be in "double quotes"**; no functions, no undefined, no comments, no trailing commas.

```
const obj = { name: "Asad", age: 20, langs: ["JS", "CSS"] };
const text = JSON.stringify(obj);                // object → JSON string
JSON.stringify(obj, null, 2);                    // pretty print with 2 spaces
const back = JSON.parse(text);                   // JSON string → object
const deepCopy = JSON.parse(JSON.stringify(obj));   // quick deep copy (loses Date, undefined, functions)
try { JSON.parse("{bad json}"); } catch (e) { console.log("Invalid JSON"); }
```

Use

JSON is used in APIs (fetch), **localStorage** (only stores strings!), config files (package.json).

15. DOM – Document Object Model (Part 1)

- The browser turns HTML into a tree of objects (the DOM). JavaScript can read, change, add and remove any element.
- ***document*** = entry point. Tree: document → html → head / body → elements → text. Node types: **element, text, comment, document.**

Selecting elements

Method	Returns
getElementById("id")	One element (or null)
querySelector("css selector")	First match ✔ (most used)
querySelectorAll("css selector")	NodeList of all matches (forEach ok)
getElementsByClassName() / ByTagName()	Live HTMLCollection (not array)
closest(sel) · matches(sel)	Nearest ancestor · does it match?

```
const title = document.getElementById("title");
const btn = document.querySelector(".btn");              // class    #id   tag   [attr]
const items = document.querySelectorAll("ul li");
items.forEach(li => console.log(li.textContent));
```

Change content, attributes & style

```
title.textContent = "New text";            // plain text (safe)
title.innerHTML = "<b>Bold</b> text";      // parses HTML (risky with user input → XSS!)
title.innerText;                           // visible text    input.value → form value

img.getAttribute("src");   img.setAttribute("alt", "Logo");   img.removeAttribute("title");
img.hasAttribute("alt");
img.src = "new.png";       link.href;      el.id = "x";       el.dataset.userId = "5";    // data-user-id

title.style.color = "red";                 // inline style (camelCase)  backgroundColor, fontSize
getComputedStyle(title).fontSize;          // final computed value
```

classList – best way to change styles

```
el.classList.add("active", "big");   el.classList.remove("active");
el.classList.toggle("dark");         // add if missing, remove if present
el.classList.contains("big");        el.classList.replace("a", "b");     el.className = "x y";
```

Create, add, remove elements

```
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);          // add at end
list.append(li);  list.prepend(li);  el.before(x);  el.after(x);        // modern methods
list.insertAdjacentHTML("beforeend", "<li>Quick</li>");   // beforebegin | afterbegin | beforeend | afterend
li.remove();                  parent.removeChild(li);       parent.replaceChild(newEl, oldEl);
const copy = li.cloneNode(true);    // true = include children
```

Navigating the tree

Speed tip: many DOM changes are slow – build a ***DocumentFragment*** / template string, insert once. Tree: ***parentElement · children · firstElementChild · lastElementChild · nextElementSibling · previousElementSibling*** · (childNodes / parentNode include text nodes).

16. DOM Events (Part 2)

An **event** is something that happens: a click, key press, page load, form submit… JavaScript listens and reacts.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", function (event) {        // BEST way (can add many listeners)
  console.log("Clicked!", event.target);
});
btn.addEventListener("click", () => alert("Hi"), { once: true });    // options: once, capture, passive
btn.removeEventListener("click", handlerFn);              // needs the SAME function reference
// Other ways (less recommended):  <button onclick="fn()">     btn.onclick = fn;
```

Common events

Type	Events
Mouse	click, dblclick, mousedown, mouseup, mouseover, mouseout, mouseenter, mouseleave, mousemove, contextmenu
Keyboard	keydown, keyup (keypress is deprecated) → **event.key**, **event.code**
Form	submit, input (every change), change, focus, blur, reset
Window / page	DOMContentLoaded, load, resize, scroll, beforeunload, online / offline
Touch / drag	touchstart, touchmove, touchend · dragstart, dragover, drop
Clipboard / media	copy, paste, cut · play, pause, ended, timeupdate

The event object

```
form.addEventListener("submit", (e) => {
  e.preventDefault();                  // stop default action (form reload, link jump)
  e.stopPropagation();                 // stop bubbling to parents
  e.target;  e.currentTarget;          // clicked element  |  element with the listener
  e.type;  e.clientX;  e.clientY;      // event type, mouse position
});
input.addEventListener("keydown", e => { if (e.key === "Enter") send(); });    // e.ctrlKey, e.shiftKey, e.altKey
document.addEventListener("DOMContentLoaded", () => { /* DOM ready */ });
```

Bubbling, capturing & delegation

Event flow

1. Capturing (window → target) · **2. Target** · **3. Bubbling** (target → window, default). A click on a child also triggers its parents' click listeners.

```
// Event delegation: ONE listener on the parent handles all children (even new ones)
document.querySelector("ul").addEventListener("click", (e) => {
  if (e.target.matches("li")) e.target.classList.toggle("done");
});
```

Working with forms

```
const form = document.forms["signup"];       // or querySelector("form")
const data = new FormData(form);             Object.fromEntries(data);      // all fields as an object
input.value;   checkbox.checked;   select.value;   radio: document.querySelector('input[name="g"]:checked').value
input.focus(); form.reset(); input.setCustomValidity("Bad!"); form.checkValidity();
```

Custom events

el.dispatchEvent(new CustomEvent("done", { detail: 5 })). Always **preventDefault()** on form submit when handling with JS.

17. Browser Object Model & Storage

BOM = objects the browser gives to talk to the browser itself. ***window*** is the global object (document, console, alert… are its properties).

Object	Use
window	innerWidth / innerHeight, scrollY, open(url), close(), scrollTo(0,0), print()
location	href, hostname, pathname, search, hash · ***location.href = "url"*** (go) · reload() · replace()
history	back(), forward(), go(-2), pushState() (change URL without reload)
navigator	userAgent, language, onLine, geolocation, clipboard, cookieEnabled
screen	width, height of the screen

Popups (blocking)

```
alert("Message");                          // OK only
const ok = confirm("Delete this?");        // OK → true, Cancel → false
const name = prompt("Your name?", "Guest");   // text or null
```

Timers

```
const id = setTimeout(() => console.log("after 2s"), 2000);   // run ONCE after delay (ms)
clearTimeout(id);                                             // cancel it
const timer = setInterval(() => console.log("tick"), 1000);   // run REPEATEDLY
clearInterval(timer);
requestAnimationFrame(draw);                                  // smooth animation (≈60fps)
```

Web Storage (store data in the browser)

	localStorage	sessionStorage	Cookies
Lifetime	Until deleted	Until tab closes	Expiry date
Size	~5 MB	~5 MB	~4 KB
Sent to server	No	No	Yes, every request

```
localStorage.setItem("theme", "dark");      localStorage.getItem("theme");      // "dark"  (null if missing)
localStorage.removeItem("theme");           localStorage.clear();              localStorage.length;
// Only STRINGS can be stored → use JSON for objects / arrays
localStorage.setItem("user", JSON.stringify({ name: "Asad" }));
const user = JSON.parse(localStorage.getItem("user") ?? "null");
```

Cookies

```
document.cookie = "username=Asad; expires=Fri, 31 Dec 2027 23:59:59 GMT; path=/; Secure; SameSite=Lax";
document.cookie;      // "username=Asad; other=value"   (read all as one string)
```

Security

Never store passwords or tokens in localStorage (any script can read it). Use **HttpOnly** cookies set by the server.

18. Error Handling & Debugging

try ... catch ... finally

```
try {
  const data = JSON.parse(text);        // code that may fail
  riskyFunction();
} catch (error) {                       // runs only if an error happens
  console.log(error.name);              // "SyntaxError"
  console.log(error.message);           // what went wrong
  console.log(error.stack);             // where it happened
} finally {
  console.log("always runs");           // cleanup: close loader, connection...
}
```

throw – create your own errors

```
function divide(a, b) {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
}
try { divide(5, 0); } catch (e) { console.error(e.message); }

class ValidationError extends Error {           // custom error class
  constructor(message, field) { super(message); this.name = "ValidationError"; this.field = field; }
}
```

Error type	When
SyntaxError	Code is written wrongly: `let 1a = 5;` · invalid JSON
ReferenceError	Variable doesn't exist / used before declaration
TypeError	Wrong type: calling a non-function, `null.name`, assign to const
RangeError	Number out of range: `new Array(-1)`, infinite recursion
URIError / EvalError	Bad URI function use / eval problems
AggregateError	Many errors together (Promise.any)

- ➜ Syntax errors are found **before** code runs; others at **runtime**. Logical errors give no message – the hardest!
- ➜ Global handlers: `window.onerror`, `window.addEventListener("unhandledrejection", ...)`. Only **handle errors you can fix.**

Debugging tools

```
console.log(x);   console.table(users);   console.dir(obj);   console.group("G"); console.groupEnd();
console.time("loop"); /* code */ console.timeEnd("loop");      // measure time
console.assert(x > 0, "x must be positive");   console.count("clicks");   console.trace();
debugger;      // pauses execution if DevTools is open
```

Browser DevTools (F12)

Console · Sources (breakpoints, step over / into) · Network (requests) · Elements · Application (storage)

Debug steps

Read the error message → find the line → console.log values → use breakpoints → fix → test again.

19. Asynchronous JavaScript

- JS is **single-threaded**: it runs one thing at a time. **Synchronous** = line by line, each waits for the previous.
- **Asynchronous** = start a long task (timer, network, file) and **continue**; handle the result later. Page never freezes

Event loop

Call Stack (runs code) ← **Microtask queue** (promises, await - runs first) ← **Callback / Task queue** (setTimeout, events). The event loop moves tasks to the stack **only when the stack is empty**. Web APIs (timers, fetch) work in the background.

```
console.log("1");
setTimeout(() => console.log("2 (timeout)"), 0);
Promise.resolve().then(() => console.log("3 (promise)"));
console.log("4");
// Output: 1, 4, 3, 2   ← sync first, then microtasks, then tasks
```

1) Callbacks (old way) → "callback hell"

```
getUser(1, (user) => { getPosts(user.id, (posts) => { getComments(posts[0], (c) => { /* pyramid of doom */
}); }); });
```

2) Promises

An object for a future value. States: **pending** → **fulfilled** (resolve) or **rejected** (reject).

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => { Math.random() > 0.5 ? resolve("Success") : reject(new Error("Failed")); }, 1000);
});
p.then(result => console.log(result))      // runs on success (can chain: return a value / promise)
 .catch(err => console.error(err))         // runs on any error in the chain
 .finally(() => console.log("done"));      // always
```

Static method	Meaning
Promise.all([p1, p2])	Wait for **all**; fails if one fails
Promise.allSettled([...])	Wait for all; gives every result / error
Promise.race([...])	First to finish (success or fail) wins
Promise.any([...])	First to **succeed**
Promise.resolve(x) · reject(e)	Ready-made promise

3) async / await ✔ (modern, looks synchronous)

```
async function load() {                    // async function always returns a Promise
  try {
    const user = await getUser(1);         // pause here until the promise is done
    const [a, b] = await Promise.all([getA(), getB()]);   // run in PARALLEL
    return user;
  } catch (err) { console.error(err); }    // errors handled with try / catch
}
const wait = ms => new Promise(res => setTimeout(res, ms));    await wait(1000);
```

Remember

await works only inside **async** functions (or top-level in modules). Awaiting in a loop runs one-by-one; use Promise.all for speed.

20. Fetch API & AJAX

→ AJAX = loading data from a server **without reloading the page**. Modern way: **fetch()** (returns a Promise). Old way: XMLHttpRequest. Data is usually **JSON**.

GET request

```
fetch("https://api.example.com/users")
  .then(response => {
    if (!response.ok) throw new Error("HTTP " + response.status);   // fetch does NOT reject on 404/500!
    return response.json();              // parse JSON body (also .text() .blob() .formData() .arrayBuffer())
  })
  .then(data => console.log(data))
  .catch(err => console.error("Network or HTTP error:", err));

// async / await version
async function getUsers() {
  try {
    const res = await fetch("/api/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const users = await res.json();
    return users;
  } catch (e) { console.error(e); }
}
```

POST / PUT / DELETE

```
const res = await fetch("/api/users", {
  method: "POST",                                    // GET, POST, PUT, PATCH, DELETE
  headers: { "Content-Type": "application/json", "Authorization": "Bearer TOKEN" },
  body: JSON.stringify({ name: "Asad", age: 20 })        // body must be a string / FormData
});
const created = await res.json();
await fetch("/api/users/5", { method: "DELETE" });
```

HTTP method	Purpose	Status code	Meaning
GET	Read data	**200 / 201**	OK / Created
POST	Create data	**301 / 304**	Redirect / cached
PUT / PATCH	Update	**400 / 401 / 403**	Bad request / login / forbidden
DELETE	Remove data	**404 / 500**	Not found / server

Useful extras

```
const params = new URLSearchParams({ q: "js", page: 2 });   fetch(`/search?${params}`);   // query string
const controller = new AbortController();                    fetch(url, { signal: controller.signal });
setTimeout(() => controller.abort(), 5000);                  // cancel request after 5s (timeout)
```

XMLHttpRequest (old way)

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/data");
xhr.onload = () => { if (xhr.status === 200) console.log(JSON.parse(xhr.responseText)); };
xhr.send();
```

CORS: browsers block requests to other domains unless the server sends the header Access-Control-Allow-Origin. Test API: jsonplaceholder.typicode.com

21. ES6+ Modern Features

Feature	Example
let / const	Block-scoped variables
Arrow functions	`const add = (a, b) => a + b`
Template literals	`` `Hi ${name}` `` multi-line, expressions
Destructuring	`const { a, b } = obj; const [x, y] = arr;`
Spread / Rest	`[...a, ...b] { ...o } function f(...args)`
Default parameters	`function f(x = 10)`
Enhanced object literals	`{ name, greet() {}, [key]: 1 }`
Classes · modules	class, extends, import / export
Promises · async / await	Async code made easy (ES2017)
for...of · Symbol · Map/Set	Iteration and new data types
Exponent **	`2 ** 10 = 1024`
Optional chaining ?.	`user?.address?.city`
Nullish coalescing ??	`value ?? "default"` (only null / undefined)
Logical assignment	`a ||= 1; b &&= 2; c ??= 3;`
Array / Object helpers	includes, flat, flatMap, at, entries, fromEntries, groupBy, toSorted
String helpers	padStart, replaceAll, trimStart, at, matchAll
BigInt · numeric separators	`123n · 1_000_000`
globalThis · structuredClone	Global object everywhere · deep copy
Top-level await · #private	await in modules · private class fields

Destructuring in depth

```
const { name: userName, age = 18, ...rest } = person;      // rename, default, rest
function show({ name, city = "Lahore" }) { }                // destructure parameters
const [a, [b, c]] = [1, [2, 3]];                            // nested
for (const { id, title } of posts) console.log(id, title);
```

Spread vs Rest (same ... different place)

Spread = expand

`Math.max(...nums)` · `[...a, ...b]` · `{ ...obj, x: 1 }`

Rest = collect

`function f(a, ...others)` · `const [x, ...tail] = arr` · `const {a, ...rest} = obj`

Symbols, WeakRef & more

```
const id = Symbol("id");    const obj = { [id]: 123 };      // unique, hidden property key
const tag = (strings, ...vals) => strings.raw.join("|");    tag`a${1}b`;      // tagged templates
const wm = new WeakMap();   Object.entries(o).map(([k, v]) => [k, v * 2]);
```

Browser support

Check **caniuse.com**; use **Babel** to convert modern JS for old browsers.

22. Prototypes & Object-Oriented Programming

OOP organises code into **objects** that combine data (properties) and behaviour (methods). 4 pillars: Encapsulation, Abstraction, Inheritance, Polymorphism.

Constructor functions

```
function Person(name, age) {          // PascalCase name, call with new
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function () {      // shared by ALL instances (saves memory)
  return `Hi, I am ${this.name}`;
};
const p1 = new Person("Asad", 20);          p1.greet();      // "Hi, I am Asad"
p1 instanceof Person                        // true          p1.constructor === Person
```

What does new do?

1. Creates an empty object.
2. Links its **__proto__** to Constructor.prototype.
3. Runs the constructor with **this** = new object.
4. Returns the object.

Prototype & prototype chain

- Every object has a hidden link ***[[Prototype]]*** (read: ***Object.getPrototypeOf(obj)*** or ***obj.__proto__***) to another object.
- When you read ***obj.prop***, JS looks in the object → its prototype → the prototype's prototype … → **null**. This is the prototype chain (inheritance in JS).

```
const animal = { eats: true, walk() { return "walking"; } };
const dog = Object.create(animal);          // dog's prototype = animal
dog.barks = true;     dog.eats;             // true (found on prototype)
dog.hasOwnProperty("eats");                 // false → it's inherited
// chain: dog → animal → Object.prototype → null          [] → Array.prototype → Object.prototype
```

Inheritance with constructor functions

```
function Student(name, age, course) {
  Person.call(this, name, age);                    // call parent constructor
  this.course = course;
}
Student.prototype = Object.create(Person.prototype);   // link prototypes
Student.prototype.constructor = Student;
Student.prototype.study = function () { return `${this.name} studies ${this.course}`; };
```

Concept	Meaning
Encapsulation	Hide internal data (closures, #private) and expose a clean interface
Inheritance	Child object reuses / extends parent's properties and methods
Polymorphism	Same method name, different behaviour (override in child)
Abstraction	Show only what is necessary, hide the complexity

23. Classes (ES6)

A **class** is a blueprint for creating objects. Syntax sugar over prototypes.

```
class Person {
  species = "human";                  // public field
  #secret = 42;                       // PRIVATE field (only inside the class)
  static count = 0;                   // static property: belongs to the class itself

  constructor(name, age) {            // runs automatically with new
    this.name = name;
    this.age = age;
    Person.count++;
  }
  greet() {                           // method (on prototype, shared)
    return `Hi, I am ${this.name}`;
  }
  get info() { return `${this.name} (${this.age})`; }      // getter  → p.info
  set info(v) { [this.name, this.age] = v.split(","); }    // setter
  #hidden() { return this.#secret; }                       // private method
  static create(n) { return new Person(n, 0); }            // static method → Person.create("A")
  toString() { return this.name; }
}
const p = new Person("Asad", 20);
p.greet();   p.info;   Person.count;   p.#secret   // SyntaxError (private)
```

Inheritance – extends & super

```
class Student extends Person {
  constructor(name, age, course) {
    super(name, age);                 // MUST call super() before using this
    this.course = course;
  }
  greet() {                           // method overriding (polymorphism)
    return super.greet() + `, I study ${this.course}`;
  }
}
const s = new Student("Ali", 19, "CS");
s.greet();   s instanceof Student;   s instanceof Person;   // true  true
```

Keyword	Meaning
class	Declares a class (not hoisted like functions; always strict mode)
constructor	Special method that initialises the new object (only one)
extends	Create a child class that inherits from a parent
super	**super(...)** calls parent constructor · **super.method()** calls parent method
static	Belongs to the class, not to instances
get / set	Computed properties with function logic
#name	Private field / method
new.target · instanceof	Detect constructor call · check type

Gotchas

Always use **new**. A class is just a function (**typeof Person === "function"**). Methods lose **this** when passed as callbacks → use arrow function or **bind**.

Mixins & composition

Object.assign(Cls.prototype, mixin) – JS has only single inheritance.

24. Modules, Iterators & Generators

ES Modules (import / export)

Split code into files. Each module has its own scope. In HTML: **<script type="module" src="main.js"></script>** (deferred, strict mode, needs a server).

```
// math.js
export const PI = 3.14;                          // named exports
export function add(a, b) { return a + b; }
export default function multiply(a, b) { return a * b; }      // ONE default export per file
// app.js
import multiply, { PI, add as sum } from "./math.js";          // default + named (rename with as)
import * as math from "./math.js";                              // everything → math.PI
const mod = await import("./heavy.js");                          // dynamic import (lazy loading)
```

- **Named export** – imported with the same name in { }. **Default export** – imported with any name, no { }. Imports are live, read-only and run once.
- Old systems: CommonJS (**require()** / **module.exports**) in Node.js, AMD.

Iterators & iterables

Iterable = has **[Symbol.iterator]()** returning an **iterator** with **next()** → **{ value, done }**. Arrays, strings, Map, Set are iterable → work with **for…of**, spread, destructuring.

```
const it = [10, 20][Symbol.iterator]();
it.next();   // { value: 10, done: false }      it.next(); // { value: 20, done: false }     it.next(); // { value:
undefined, done: true }

const range = {                                  // custom iterable
  from: 1, to: 3,
  [Symbol.iterator]() { let n = this.from, end = this.to;
    return { next: () => n <= end ? { value: n++, done: false } : { done: true } }; }
};
[...range];      // [1, 2, 3]
```

Generators – functions that can pause (function*)

```
function* count() {
  yield 1;                  // pause & give a value
  yield 2;
  return 3;
}
const g = count();
g.next();  // {value: 1, done: false}      g.next(); // {value: 2, done: false}      g.next(); // {value: 3, done: true}

function* idMaker() { let id = 1; while (true) yield id++; }     // infinite sequence (lazy!)
function* all() { yield* [1, 2]; yield* count(); }               // yield* delegates to another iterable
```

Why generators?

Lazy values, infinite lists, custom iteration, and the base idea of async/await.

Async iteration

async function* and **for await (const x of stream)** – read data chunks over time.

Symbol well-known uses

Symbol.iterator (for…of) · **Symbol.asyncIterator** · **Symbol.toPrimitive** · **Symbol.hasInstance** · **Symbol.for("key")** (global registry).

25. Regular Expressions (RegExp)

A regex is a pattern to search, match and replace text. Written between slashes: **/pattern/flags** or **new RegExp("pattern", "flags")**.

Flag	Meaning	Flag	Meaning
g	global – find all matches	m	multi-line (^ $ per line)
i	ignore case	s	dot matches new line
u	Unicode support	y	sticky (from lastIndex)

Pattern syntax

Pattern	Meaning	Pattern	Meaning
.	any character	\d \D	digit / non-digit
^ $	start / end of text	\w \W	word char [A-Za-z0-9_]
* + ?	0+ · 1+ · 0 or 1	\s \S	whitespace / not
{3} {2,5}	exact / range count	\b	word boundary
[abc] [^abc]	one of / none of	( ) (?:)	capture / non-capture group
[a-z0-9]	range	a|b	a OR b
(?=x) (?!x)	lookahead / negative	(?<=x) (?<!x)	lookbehind / negative
(?<name>...)	named group	*? +?	lazy (shortest) match

Methods

```
const re = /\d{3}-\d{4}/;
re.test("Call 555-1234");                  // true / false
"Call 555-1234".match(re);                  // ["555-1234", index: 5, ...]   with /g → array of all matches
[..."a1b22".matchAll(/\d+/g)];              // iterate all matches with details
"Call 555-1234".search(re);                 // 5 (index or -1)
"a-b-c".replace(/-/g, "+");                // "a+b+c"        "john smith".replace(/\b\w/g, c => c.toUpperCase());  // Title Case
"a, b;c".split(/[,;]\s*/);                 // ["a","b","c"]
const m = /(?<year>\d{4})-(?<month>\d{2})/.exec("2026-10");    m.groups.year;    // "2026"
```

Useful patterns

```
const email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;                  // simple email check
const phonePK = /^(\+92|0)3\d{9}$/;                          // Pakistani mobile number
const strongPw = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;    // 8+ chars, lower, upper, digit
const url = /^https?:\/\/[\w.-]+\.[a-z]{2,}(\/\S*)?$/i;
const spaces = /\s+/g;    str.replace(spaces, " ");          // collapse many spaces
```

Remember

Escape special characters with \ : \. * \? \(\/. Without g, replace() changes only the first match.

Practice

Test patterns on **regex101.com**. Keep regexes simple and readable!

26. Advanced Concepts

Execution context & call stack

- When code runs, JS creates an **execution context**: ① **Creation phase** (memory is set up: variables hoisted, functions stored, ***this*** decided) ② **Execution phase** (code runs line by line).
- The **call stack** is a LIFO list of running functions. Too deep recursion → ***RangeError: Maximum call stack size exceeded*** (stack overflow).

Scope chain & lexical environment

A function looks for a variable in its own scope → outer scope → … → global. This chain is fixed where the function is **written** (**lexical scoping**) – that is why closures work.

Debounce & throttle – performance patterns

```
function debounce(fn, delay) {                  // run only AFTER the user stops (search box)
  let timer;
  return (...args) => { clearTimeout(timer); timer = setTimeout(() => fn(...args), delay); };
}
function throttle(fn, limit) {                  // run at most once per time period (scroll / resize)
  let wait = false;
  return (...args) => { if (!wait) { fn(...args); wait = true; setTimeout(() => (wait = false), limit); } };
}
input.addEventListener("input", debounce(search, 300));
```

Immutability, shallow vs deep copy

```
const a = { x: 1, inner: { y: 2 } };
const shallow = { ...a };              shallow.inner.y = 99;       // a.inner.y also changed!
const deep = structuredClone(a);     // safe deep copy  (JSON trick loses functions / Dates)
Object.freeze(a);                    // shallow freeze - nested objects stay mutable
```

Memory management & garbage collection

- JS frees memory automatically using **mark-and-sweep**: objects that can't be reached from roots are removed.
- **Memory leaks:** forgotten timers (setInterval), unremoved event listeners, global variables, detached DOM nodes, closures holding big data. Fix: clearInterval, removeEventListener, use WeakMap / WeakRef.

More advanced topics

Topic	Idea
Strict mode	"use strict" – no undeclared variables, no duplicate params, this = undefined in functions
Proxy & Reflect	**new Proxy(target, { get(t, k) { … } })** – intercept reads / writes (validation, reactivity)
Event loop phases	Sync code → microtasks (promises, queueMicrotask) → render → macrotasks (timers, events)
Web Workers	Run heavy JS on another thread so the page doesn't freeze
Pure functions & FP	map / filter / reduce, immutability, composition, currying
Design patterns	Module, Singleton, Observer (events), Factory, Pub/Sub
TypeScript	JavaScript + static types (compiles to JS) – popular in big projects

Performance tips

Avoid layout thrashing • cache DOM lookups • use event delegation • lazy load • minify & bundle • don't block the main thread.

27. Web APIs (Browser features)

Browsers give JavaScript extra powers through APIs. All are asynchronous or event-based; many ask the user for **permission** and need **HTTPS**.

Geolocation

```
navigator.geolocation.getCurrentPosition(
  pos => console.log(pos.coords.latitude, pos.coords.longitude),
  err => console.log(err.message), { enableHighAccuracy: true });
const watch = navigator.geolocation.watchPosition(cb);    navigator.geolocation.clearWatch(watch);
```

Clipboard & Notifications

```
await navigator.clipboard.writeText("Copied!");          const t = await navigator.clipboard.readText();
if (await Notification.requestPermission() === "granted") new Notification("Hello!", { body: "Message" });
```

IntersectionObserver – detect when an element is visible (lazy load, infinite scroll)

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(e => { if (e.isIntersecting) { e.target.classList.add("show"); observer.unobserve(e.target); } });
}, { threshold: 0.3 });
document.querySelectorAll(".card").forEach(c => observer.observe(c));
```

Canvas, Web Workers, History

```
const ctx = canvas.getContext("2d");   ctx.fillStyle = "orange";   ctx.fillRect(10, 10, 100, 50);   ctx.arc();
ctx.fillText();
const worker = new Worker("worker.js");   worker.postMessage(data);   worker.onmessage = e =>
console.log(e.data);   // in worker: self.onmessage
history.pushState({ page: 2 }, "", "/page2");   addEventListener("popstate", e => render(e.state));   // SPA
routing
```

API	Purpose
MutationObserver / ResizeObserver	Watch DOM changes / element size changes
Drag & Drop API	dragstart, dragover, drop + **e.dataTransfer**
File API / FileReader	Read user-selected files: **input.files[0]**, readAsText, readAsDataURL
IndexedDB	Big database in the browser (structured data, offline apps)
Service Worker / Cache API	Offline support, PWA, background sync, push
WebSocket	Real-time two-way connection: **new WebSocket("wss://...")** (chat, live data)
Fullscreen / Page Visibility	el.requestFullscreen() • document.hidden
Web Audio / Speech / MediaDevices	Sound • voice recognition / synthesis • camera & microphone (getUserMedia)
Share / Vibration / Battery	navigator.share() • navigator.vibrate(200) • device info
Intl / URL / structuredClone	Formatting dates, numbers • URL parsing • deep copy

Feature detection

Check before using: **if ("geolocation" in navigator) { ... }**

28. Node.js, npm & Tools

Node.js

- Node.js runs JavaScript **outside the browser** (Chrome's V8 engine + libuv). Used for servers, APIs, tools, CLIs. Non-blocking I/O, great for real-time apps.
- No ***window / document***; has ***global***, ***process***, ***require***, built-in modules: **fs** (files), **http**, **path**, **os**, **events**, **crypto**.

```
// app.js  →  run with:  node app.js
const fs = require("fs");                          // CommonJS    (or: import fs from "fs";  with "type":"module")
const text = fs.readFileSync("data.txt", "utf8");
console.log(process.argv, process.env.PORT);

const http = require("http");
http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Node!");
}).listen(3000);                                   // open http://localhost:3000
```

Express.js (popular framework)

```
const express = require("express");
const app = express();
app.use(express.json());                                    // middleware: parse JSON body
app.get("/api/users", (req, res) => res.json([{ id: 1 }]));
app.post("/api/users", (req, res) => res.status(201).json(req.body));
app.listen(3000, () => console.log("Server running"));
```

npm – Node Package Manager

Command	Does
npm init -y	Create package.json (project info + dependencies)
npm install express	Install a package into node_modules (add to dependencies)
npm i -D eslint	Install as dev dependency (only for development)
npm install	Install everything listed in package.json
npm run dev / start / build	Run a script from package.json "scripts"
npm uninstall · update · audit	Remove · update · check security
npx create-vite@latest	Run a package without installing it

- **package.json** = project file · **package-lock.json** = exact versions · **node_modules** = packages (never upload; add to .gitignore) · semantic versions: **^1.2.3** (minor updates ok), **~1.2.3** (patch only).

Tools & ecosystem

- **Git & GitHub** – version control
- **ESLint** – finds code problems; **Prettier** – formats code
- **Babel** – converts modern JS for old browsers
- **Vite / Webpack** – bundlers
- **Jest / Vitest** – testing
- **TypeScript** – typed JS
- **React, Vue, Angular, Svelte** – frameworks
- **Next.js, Nuxt** – full-stack frameworks

Roadmap: **JS Basics** → **DOM & Events** → **Async & Fetch** → **ES6+** → **Git, npm** → **React** → **Node & DB**

29. Mini Projects (Practice)

Project 1: Counter

counter.html

```html
<h1 id="count">0</h1>
<button id="inc">+</button> <button id="dec">-</button> <button id="reset">Reset</button>
<script>
  let count = 0;
  const show = document.querySelector("#count");
  const update = () => (show.textContent = count);
  document.querySelector("#inc").addEventListener("click", () => { count++; update(); });
  document.querySelector("#dec").addEventListener("click", () => { count--; update(); });
  document.querySelector("#reset").addEventListener("click", () => { count = 0; update(); });
</script>
```

Project 2: To-Do list with localStorage

todo.html

```html
<input id="task" placeholder="New task"> <button id="add">Add</button>
<ul id="list"></ul>
<script>
  let todos = JSON.parse(localStorage.getItem("todos") ?? "[]");
  const list = document.querySelector("#list");
  function render() {
    list.innerHTML = "";
    todos.forEach((t, i) => {
      const li = document.createElement("li");
      li.textContent = t.text;
      if (t.done) li.style.textDecoration = "line-through";
      li.onclick = () => { t.done = !t.done; save(); };          // toggle done
      const del = document.createElement("button");
      del.textContent = "x";
      del.onclick = (e) => { e.stopPropagation(); todos.splice(i, 1); save(); };
      li.append(del);  list.append(li);
    });
  }
  function save() { localStorage.setItem("todos", JSON.stringify(todos)); render(); }
  document.querySelector("#add").onclick = () => {
    const input = document.querySelector("#task");
    if (input.value.trim()) { todos.push({ text: input.value.trim(), done: false }); input.value = ""; save(); }
  };
  render();
</script>
```

Project 3: Fetch data from an API

```html
async function loadPosts() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts?_limit=5");
  const posts = await res.json();
  document.querySelector("#out").innerHTML = posts.map(p => `<h3>${p.title}</h3><p>${p.body}</p>`).join("");
}
loadPosts();
```

More ideas

① Calculator ② Digital clock & stopwatch ③ Quiz app ④ Image slider ⑤ Password generator ⑥ Weather app (API) ⑦ Dark mode toggle ⑧ Form validation ⑨ Tic-Tac-Toe ⑩ Typing speed test

30. Interview Q&A – Part 1

Q1. What is JavaScript?

A high-level, interpreted, single-threaded, dynamically typed language used to make web pages interactive; also runs on servers (Node.js).

Q2. Difference between var, let and const?

var: function-scoped, hoisted (undefined), re-declarable. let: block-scoped, TDZ, re-assignable. const: block-scoped, cannot be re-assigned (object contents can change).

Q3. == vs ===?

== compares after type conversion (0 == "0" true); === compares value and type (0 === "0" false). Prefer ===.

Q4. What are the data types?

Primitives: string, number, bigint, boolean, undefined, null, symbol. Non-primitive: object (arrays, functions, dates...). undefined = no value yet; null = empty on purpose (typeof null is "object", an old bug).

Q5. What is hoisting?

Declarations are moved to the top of their scope at compile time. Function declarations are fully hoisted; var is hoisted as undefined; let / const are in the temporal dead zone.

Q6. What is a closure?

A function that remembers variables from its outer scope even after the outer function has returned. Used for private data, counters, memoization.

Q7. What is the value of this?

Depends on how a function is called: method → the object; plain function → window / undefined; arrow → lexical this; new → the new object; call / apply / bind → chosen value (call(this, a, b); apply(this, [a, b]); bind returns a new function).

Q8. Arrow vs normal functions?

Arrow: shorter, no own this / arguments / prototype, can't be a constructor, lexical this.

Q9. map vs forEach vs filter vs reduce?

forEach just loops (returns undefined); map returns a transformed array; filter returns matching items; reduce combines to one value.

Q10. What is the event loop?

Mechanism that lets single-threaded JS be asynchronous: when the call stack is empty it takes tasks from the microtask queue (promises) first, then the task queue (timers, events).

Q11. What are promises? async / await?

Promise = object for a future value (pending, fulfilled, rejected) with then / catch / finally. async / await is syntax sugar that makes promise code look synchronous.

Q12. Shallow vs deep copy?

Shallow ({...obj}, slice) copies only the first level; nested objects are shared. Deep (structuredClone, JSON trick) copies everything.

Q13. What is event bubbling / delegation?

Events travel from the target up to its parents (bubbling). Delegation attaches one listener to a parent and uses event.target to handle children.

30. Interview Q&A – Part 2

Q14. What is the prototype chain?

Every object links to a prototype; property lookup walks up this chain until null. It is how inheritance works in JS.

Q15. Class vs constructor function?

Class is cleaner syntax over the prototype system; has constructor, extends, super, static, private fields; always strict and not hoisted.

Q16. What is the difference between localStorage, sessionStorage and cookies?

localStorage: permanent ~5MB; sessionStorage: until tab closes; cookies: ~4KB, sent with every request, can expire and be HttpOnly.

Q17. What is debouncing and throttling?

Debounce runs a function after the user stops triggering it; throttle runs it at most once per interval.

Q18. What is a pure function?

Same input gives same output and no side effects (no changing outside data).

Q19. What is destructuring, spread and rest?

Destructuring unpacks values into variables; spread (...) expands iterables / objects; rest (...) collects remaining items into an array.

Q20. What is the difference between synchronous and asynchronous code?

Sync blocks until done; async starts a task and continues, handling the result later via callbacks / promises.

Q21. What is event.preventDefault() and stopPropagation()?

preventDefault stops the browser's default action (form submit, link). stopPropagation stops the event from bubbling up.

Q22. What is JSON? stringify vs parse?

Text format for data. JSON.stringify(obj) → string; JSON.parse(string) → object.

Q23. What is NaN? How to check it?

Not-a-Number result of invalid math. NaN !== NaN, so use Number.isNaN(x).

Q24. Difference between for...in and for...of?

for...in loops over keys (enumerable properties); for...of loops over values of iterables (arrays, strings, Maps, Sets).

Q25. What are template literals? Optional chaining? Nullish coalescing?

Backtick strings with ${}; ?. safely accesses nested properties; ?? uses the default only for null / undefined.

Q26. What is a memory leak? How to avoid?

Memory that is kept though no longer needed (timers, listeners, globals, closures). Clear timers, remove listeners, avoid globals.

Q27. What is strict mode? CORS? XSS?

Strict mode = safer JS rules. CORS = browser rule for cross-domain requests. XSS = injecting malicious scripts (avoid innerHTML with user data).

Q28. Quick output questions

```
console.log(typeof null);          // "object"      console.log(0.1 + 0.2 === 0.3);   // false
console.log([] + []);              // ""            console.log("5" + 3, "5" - 3);   // "53"  2
console.log([1,2,3] == "1,2,3"); // true            console.log(NaN === NaN);         // false
```

31. Quick Cheat Sheet

Basics

Types: string number bigint boolean undefined null symbol object
Variables: const (default) • let • avoid var
Compare: use === and !==

Falsy values

false, 0, -0, 0n, "", null, undefined, NaN – everything else is truthy.

String

length, slice, substring, indexOf, includes, startsWith, endsWith, replace, replaceAll, split, trim, toUpperCase, toLowerCase, padStart, repeat, at, match

Array

change: push pop shift unshift splice sort reverse
no change: slice concat join indexOf includes flat
loops: forEach map filter reduce find some every

Object

Object.keys / values / entries / assign / freeze / fromEntries • {...o} • destructuring • ?. • ??

Math & Number

Math.round floor ceil trunc abs max min pow sqrt random • parseInt parseFloat toFixed Number.isNaN

DOM

querySelector(All) • getElementById • createElement • append • remove • textContent • innerHTML • classList.add/remove/toggle • setAttribute • style • dataset

Events

addEventListener(type, fn) • click input change submit keydown scroll load DOMContentLoaded • preventDefault • stopPropagation • target • delegation

Async & Browser

```
setTimeout(fn, ms);  setInterval(fn, ms);                    // timers
const res = await fetch(url);  const data = await res.json();    // fetch + async / await
Promise.all([p1, p2]);   p.then(f).catch(e).finally(g);          // promises
localStorage.setItem(k, JSON.stringify(v));   JSON.parse(localStorage.getItem(k));
try { ... } catch (e) { ... } finally { ... }    throw new Error("msg");
```

Loops & functions

```
for (let i = 0; i < n; i++) {}   for (const x of arr) {}   for (const k in obj) {}   while (cond) {}
function f(a, b = 1, ...rest) {}   const g = (a) => a * 2;   (() => {})();   class A extends B { constructor() { super(); } }
```

Remember

Practice daily • read errors carefully • build small projects • use console.log & DevTools • read MDN docs (developer.mozilla.org) • then learn **React** & **Node.js**. 💪

~ Thank you! Keep learning ~

by asadokz18 ✿